

Unit 3— Exercises — Amortized Analysis

CS 201 - Data Structures II

Spring 2024

Habib University

Syeda Saleha Raza

Exercise – Amortized analysis of Queue using two stacks

We know that a queue (FIFO) can be implemented using two stacks (LIFO). Say that our stack has three operations, push, pop and isEmpty, each with cost 1. The queue can be implemented as follows:

- enqueue(x): push x onto stack1
- dequeue(): if stack2 is empty then pop the entire contents of stack1 pushing each element in turn onto stack 2. Now pop from stack2 and return the result.
- Worst Case: ?
- Let's analyse its amortized complexity.

Exercise – Amortized analysis of Queue using two stacks

- Considering that,
 - Each elements is pushed at most two times.
 - Each elements is popped at most two times.
- Applying accounting method, what cost should each enqueue() operation be charged to pay for subsequent push()/pop() operations on dequeue().

Exercise - Multipop

- You are working with a stack that has an additional functionality of `multipop(k)`. `multipop(k)` will either pop the top k elements in the stack, or if it runs out of elements before that, it will pop all of the elements in the stack and stop. The pseudo-code for `multipop(k)` would look like this:

```
multipop(k):  
    while stack not empty and  $k > 0$ :  
         $k = k - 1$   
        stack.pop()
```

Exercise - Binary k-bit Counter

Q2 – A binary k-bit counter can be implemented with a k-element binary array that can count up to n. The counter is initially 0. The only operation is increment(A), which adds 1 to the current number in the counter, as shown below:

Count	A[4]	A[3]	A[2]	A[1]	A[0]
0	0	0	0	0	0
1	0	0	0	0	1
2	0	0	0	1	0
3	0	0	0	1	1
4	0	0	1	0	0
5	0	0	1	0	1
6	0	0	1	1	0
7	0	0	1	1	1
8	0	1	0	0	0
9	0	1	0	0	1

Increment():

```
i = 0
while i < A.length and A[i] == 1:
    A[i] = 0
    i = i + 1
if i < A.length:
    A[i] = 1
```

Exercise - Stop-min-stack

Problem 1

(20 points) [**Amortized Analysis**] Let us define a **stop-min-stack** with a stack as the underlying data structure, which stores a sequence of items and has the following operations:

1. `push(x)` adds the item `x` to the top of the underlying stack.
2. `peek()` returns the item at the top of the underlying stack without removing it.
3. `stop-min-pop()` pops a local minimum from the existing items in the stack. This operation works as follows:
 - (a) removes an existing item from the stack using `pop()`, (if the stack was empty before the `pop()`, `NULL` is returned; if the stack is empty as a result of the `pop()`, the popped item is returned)
 - (b) tests whether the popped item is greater than or equal to the current item at the top of the underlying stack using `peek()`.
 - (c) If yes, the recently popped item is discarded.
 - (d) Steps (a) - (c) are repeated as a loop until the recently popped item is less than the topmost item in the stack or if the underlying stack is empty.
 - (e) Finally, the recently popped item is returned.

Exercise - Beach View

- Karachi has many tall buildings, but only some of them have a clear view of Arabian sea. Suppose we are given an array $A[1 .. n]$ that stores the height of n buildings on a city block, indexed from west to east. Building i has a good view of the sea if and only if every building to the east of i is shorter than i .
- Here is an algorithm that computes which buildings have a good view. What is the running time of this algorithm? Perform amortized analysis.

```
GOODVIEW( $A[1 .. n]$ ):  
  initialize a stack  $S$   
  for  $i \leftarrow 1$  to  $n$   
    while ( $S$  not empty and  $A[i] > A[\text{TOP}(S)]$ )  
      POP( $S$ )  
    PUSH( $S, i$ )  
  return  $S$ 
```

Thanks